

Single Agent Systems & Agent Pipelines - Quiz (Week 8)

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

A stateful directed graph is a workflow where each step, called a node, can remember information from previous steps and use it while making decisions. This allows the agent to choose different paths depending on the current state or input. It supports branching, looping, and dynamic execution, making the workflow more flexible. In contrast, a simple linear pipeline always follows a fixed sequence of steps from start to finish without changing its path or storing previous information. Stateful graphs are therefore more suitable for building intelligent and adaptive AI agents.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

In an agent workflow, nodes represent individual tasks or operations performed by the agent, such as processing a query, calling a tool, or generating a response. Edges define the connection between these nodes and determine the order in which tasks are executed. Together, they form the complete workflow of the agent. For example, a Calculator Tool can be considered a node, while the connection that routes a mathematical query from the input node to the calculator node is an edge.

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Conditional routing is the process of analyzing a user's request and directing it to the most appropriate tool or function based on predefined conditions. It helps the agent make decisions instead of following the same workflow for every query. A simple routing logic can be:

- If the query contains "calculate", send it to the Calculator Tool.
 - If the query contains "keywords", send it to the Keyword Extraction Tool.
 - For all other queries, return a general conversational response.
- This approach makes the agent more efficient and organized.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Cycles or loops allow an agent to repeat a task until it is completed successfully or until a stopping condition is reached. This makes the system more reliable when dealing with temporary failures or incomplete results. For example, if an agent tries to fetch data from an external API and the request fails because of a network issue, it can retry the request instead of immediately stopping. Retry loops improve the overall robustness and reliability of agent-based systems.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

A single-agent system can simulate multi-agent behavior by dividing its work into multiple logical stages. It can first understand the user's query, then decide which tool to use, execute that tool, and finally generate the response. Although only one agent is running, it performs different

responsibilities one after another, creating the effect of multiple specialized agents working together. This approach simplifies implementation while still handling complex tasks efficiently.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

JSON schema tools define a standard structure for the data exchanged between an agent and its tools. They specify required fields, expected data types, and the format of inputs and outputs. Using a predefined schema ensures that all components communicate consistently and reduces the chances of invalid or incomplete data. It also makes debugging and integration easier because every tool follows the same structured format.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential tool calls execute one task after another, where each step may depend on the result of the previous step. This approach is useful when the workflow has dependencies between tasks. Parallel tool calls execute multiple independent tasks simultaneously, reducing the total execution time. Parallel execution is preferred when tasks do not rely on each other's outputs, while sequential execution is better when one task must finish before the next can begin.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

Error handling is important to ensure that an agent continues to work even when unexpected situations occur. One common strategy is using **try-except blocks** to catch exceptions and return meaningful error messages instead of crashing. Another strategy is implementing a **retry mechanism** for temporary issues such as network failures or unavailable services. Maintaining logs of errors is also useful because it helps developers identify problems and improve the system over time.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation examines the complete sequence of actions taken by an agent while solving a task rather than only checking the final answer. It evaluates how the agent reasoned, which tools it selected, and whether each decision was appropriate. This provides better insights into the agent's behavior and helps identify mistakes that may not be visible from the final output alone. It is useful for debugging, improving decision-making, and optimizing overall agent performance.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task completion rate measures the percentage of tasks successfully completed by an agent without errors. Cost metrics measure the resources used during execution, such as API calls, response time, computational resources, or operational cost. These metrics can be tracked by monitoring multiple executions over time. To improve performance, developers can optimize routing logic, reduce unnecessary tool calls, improve decision-making, and select efficient algorithms. Balancing high task completion with low operational cost is important for building scalable and reliable AI systems.